

System Latencies

This report describes the sources and magnitudes of end-to-end latencies in a “strap-down” multi-camera infrared viewing system for deployment in a rotary-wing aircraft. It describes approaches to overcoming these latencies and describes the latency for three potential display solutions.

The chosen scenarios to be optimized include:

1. Flying fast and high, far from any nearby objects.
2. Navigating slowly among buildings searching for a target.

Because the backgrounds in these scenarios are largely unmoving, self-motion latency will dominate. The main impact of self-motion latency is “swimming” and distortion of the world around the pilot due to aircraft and head rotation between the time light enters a pixel on the camera until it exits the display. The primary impact of this latency is to induce pilot discomfort and (at levels exceeding several tens of milliseconds), sickness and reduction in task performance.

Therefore, the system is being designed to **minimize self-motion introduced by latency**. This requires two applications of the standard latency-reduction techniques used in virtual-reality (VR) displays; once to adjust for the motion during camera capture and scan-in and again to adjust for the motion during rendering and display scan-out. Given accurate measurements and a calibrated system, these can essentially remove motion and keep the world steady.

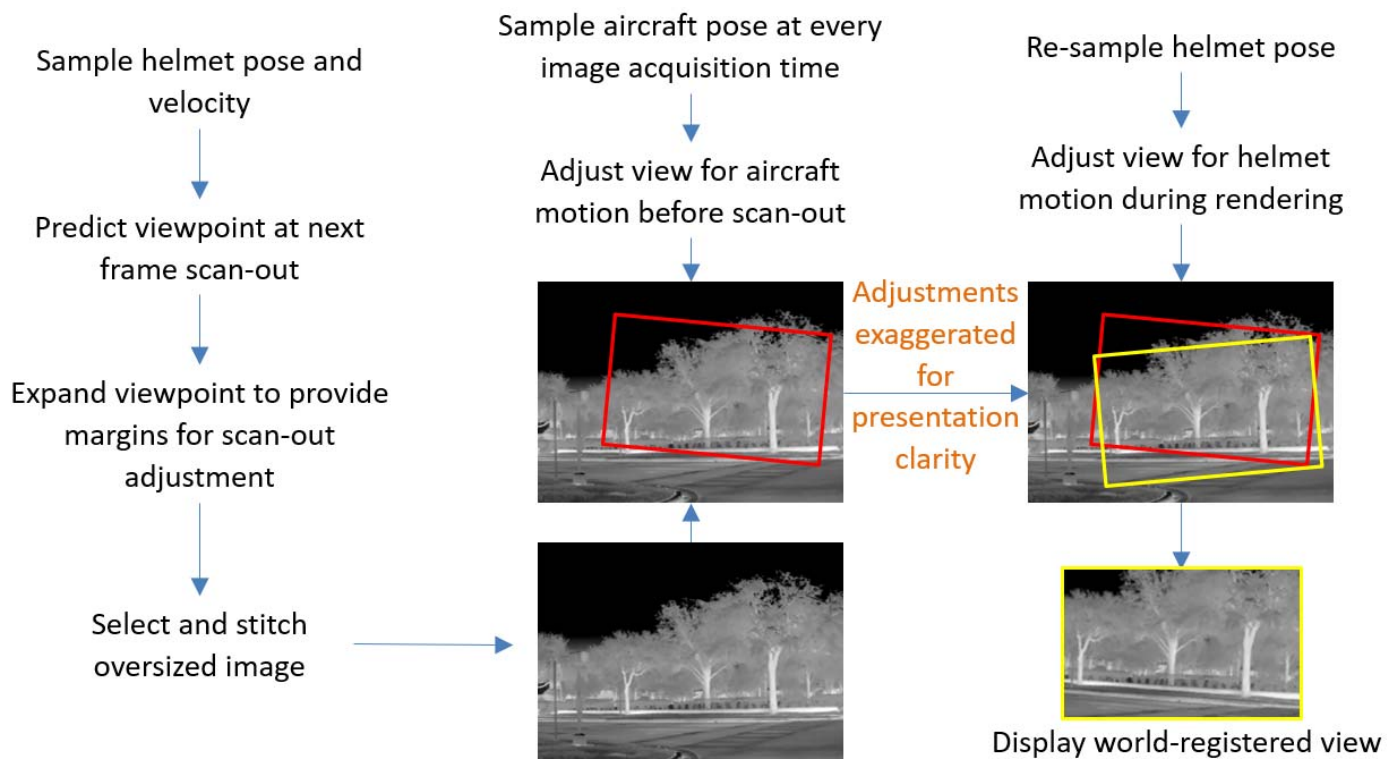


Figure 1: Removal of latency-induced self-motion: Two sequential operations of standard virtual-reality techniques for latency compensation can remove self-motion artifacts. The first compensates for helicopter motion during camera acquisition and scan-in, and the second for helicopter and pilot head motion during rendering and display scan-out.

Summary

The latency characteristics of three potential display solutions were studied, based on end-to-end system latencies including internal latency, photon latency and uncorrectable self-motion latency (which causes sickness). Details on each system are listed after the latency analysis and the performance of each is summarized here.

Display solution	Internal latency	Average photon latency	Uncorrectable self-motion latency
Directly driven monitor or HMD	19-41ms	29-59ms	0ms
Elbit Xsight lab kit	35-71ms	45-89ms	0ms
Display per sensor	0.01ms	10-18ms	10-18ms

Recommendations: A desktop monitor or commercial head-mounted display can be made to largely correct self-motion latency. By working with Elbit, it was possible to do the same with the Xsight lab kit. Even though it has low latency, the display-per-sensor solution is not viable due to its intrusive nature.

Latencies

There are multiple ways to describe latency and the effects that it produces within a system. In this report we use the following terminology:

- **Internal latency:** The duration of time from when the first pixel of an image comes into the system from a camera scan-out until the last pixel is scanned out to the display. This is a typical system specification.
- **Photon latency:** The duration of time from when a photon arrives at a sensor until the corresponding photon leaves a display. Some photon latency is unavoidable due to sensor integration and display persistence times.
- **Self-motion latency:** This includes artificial pixel offsets caused by motion of the aircraft or user's head and eyes during photon latency. This is the type of latency that causes "simulation sickness" in virtual-reality systems and wobbling virtual objects in augmented-reality systems. These pixel offsets cause the world to "swim" around the head or aircraft, and they cause shearing, stretching, and squashing of objects due to sensor and display scanning characteristics. To a large degree, *these effects can be modeled and removed (including sensor-integration and display-persistence photon latency)*. This latency is composed of two components:
 - **Rotational:** In slowly moving situations, rotation of the aircraft and head will cause the largest pixel offsets. Modeling them requires having an accurate estimate of aircraft and head rotation (available from readily available rotational sensors) and accurate measurement of the relative timing of pixel arrival and display. For rotation about the center of projection, the pixel offsets do not depend on the distance of the objects in the scene from the sensor. For systems whose sensor center of projection and display center of projection differ, they do depend on object distance from the sensor.
 - **Translational:** For moving aircraft, the pixel offset caused by self-motion latency *depends on the distance from the sensor to the object in the scene* that it is viewing. This is related to perspective projection, where near objects appear to be larger than far objects.
- **Scene-motion latency:** This includes artificial pixel offsets caused by objects within the scene moving or changing brightness over time. This is unavoidable for instantaneous flashes and randomly moving objects. Removing latency for objects in smooth motion can be done by constructing a model of the scene and its separate objects and moving the pixels associated with an object over time. The amount of pixel offset is related to the distance and speed of motion of an object.

To understand the differences between these latencies, consider the following scenario: **Interactive 3D replay:** A set of video cameras attached to an aircraft record as it is flown on a mission. Aircraft motion and rotation are simultaneously recorded. An interactive viewer using a head-mounted display wants to study the mission. The photon latency is very high (the video was taken perhaps weeks or even years in the past) and the scene-motion latency is very high (the moving objects could be anywhere by the time the replay is happening), but the rotational self-motion latency from the

camera point of view can be made very low as described in the *Avoiding Sickness* section below. This provides a distortion-free view of the mission, enabling freezing time and looking around or playing forward and watching details of past actions as they happened to the front or either side of the aircraft. The participant can look in any direction covered by the cameras at any time in the video and the environment will remain stable and clear. The motion is clear and in context and the experience does not induce sickness.

Components of Photon Latency

Camera

Integration delay: Each line in the camera begins integration slightly later than the line before, but each integrates for the same length of time. All photons that arrive within the integration integral are counted the same when scanned out, even though they arrived across the entire interval. The average delay for a given photon in the interval is half of the interval. For an *integration interval with a 20ms time constant, this is ~10ms*.

Scan out: Each line is buffered and then read using analog-to-digital conversion. Its values are sent via a transmission line to a receiver. The A/D and transmission rates are usually well matched to the frame rate. This delay is much shorter than integration delay because it is divided by the number of lines in the image. For a 60fps scan out of a 1920x1080 sensor, this is *~0.008ms*.

Buffering

Because the rendering system must scan out to a raster display in a fixed order (left-to-right, top-to-bottom is common) and because each line from each camera comes in at different times, and because rotation of the head can cause any line from any camera to end up at any location in the display, the entire frame from each camera must be buffered in GPU memory so that it can be transformed as a whole image for display. (This may also be needed for the removal of self-motion artifacts from cameras due to motion of the aircraft, but the same buffer can be used for all portions of the rendering pipeline.)

The latency for the last line in the buffer is zero and the latency for the first is a whole frame time, with the average latency for a line being half the frame time. For a *60fps scan rate, this is ~8ms*.

Transmission

The subset of video streams that are within the field of view of the display (plus a small border for timing fix-up) must be transmitted to the texture memory of the GPU that drives the display. Cameras whose views are more peripheral may be down-sampled to a lower resolution if we're using a display with a wide field of view.

Streaming to texture memory can be done progressively, with portions of the different images sent separately and interleaved. Because we've already separately counted the GPU buffering delay, we do not need to count it again so long as the rest of the system performs its operations in a streaming fashion – sending each line (or down-sampled set of lines) as soon as the line is received.

Assuming that the throughput of the transmission system is sufficient to stream the maximum required number of videos at full rate, the transmission delay will be the time taken for a single line. For a 60fps scan out of a 1920x1080 sensor, this is *~0.008ms*. Even sending 100 lines at a time to reduce overhead makes this *~0.8ms*.

Rendering

Rendering as described here includes all stages of the pipeline that transform images, including any depth estimation and adjustment of input images for translation or rotation of the aircraft during capture along with combining videos into the common output buffer that will be sent to the display.

Rendering rates depend greatly (by a factor of 20+) on the GPU being used (embedded vs. laptop vs. server). Determining actual times will require performance optimization and measurement. The estimates provided below are not actual measurements and are based on an nVidia GeForce 1080, which is a high-end, 3-generation-old desktop server card.

Depth estimation: The delay for estimating the depth depends on the available hardware and algorithms being used. For LIDAR-based systems, this will be very low. For global-shutter cameras, multi-depth-rendering techniques are available that might take ~5ms. For rolling-shutter cameras, more complex methods must be developed and evaluated. An initial implementation of rolling-shutter reconstruction runs asynchronously from the rendering loop at 30fps on two pairs of cameras and requires adding ~6ms to the rendering latency to update the projection geometry based on the depth estimations.

Camera motion correction: When using algorithms like those in modern VR systems, motion is assumed to be constant over the frame time and the correction is done by modifying the matrices used for composition. This takes no additional per-pixel time and merely requires updating the matrix, which takes **~0.1ms**.

Composition: The depth- and motion-corrected camera images are rendered into a world-space scene that consists of a triangle-mesh object that has many fewer vertices than pixels in the image. One such rendering is done for each visible camera, putting them into a common “world space”. The rendering time depends on the complexity of the geometry and number of textures and is expected to take **~1-5ms**.

Head motion correction: When using algorithms like those in modern VR systems, motion is assumed to be constant over the frame time and the correction is done by modifying the matrices used for rendering to the display. This takes no additional per-pixel time and merely requires updating the matrix, which takes **~0.1ms**.

Rendering to the display: The 3D scene constructed during composition is rendered from one or two viewpoints. The rendering time depends on the complexity of the geometry and number of textures and is expected to take **~1-5ms**.

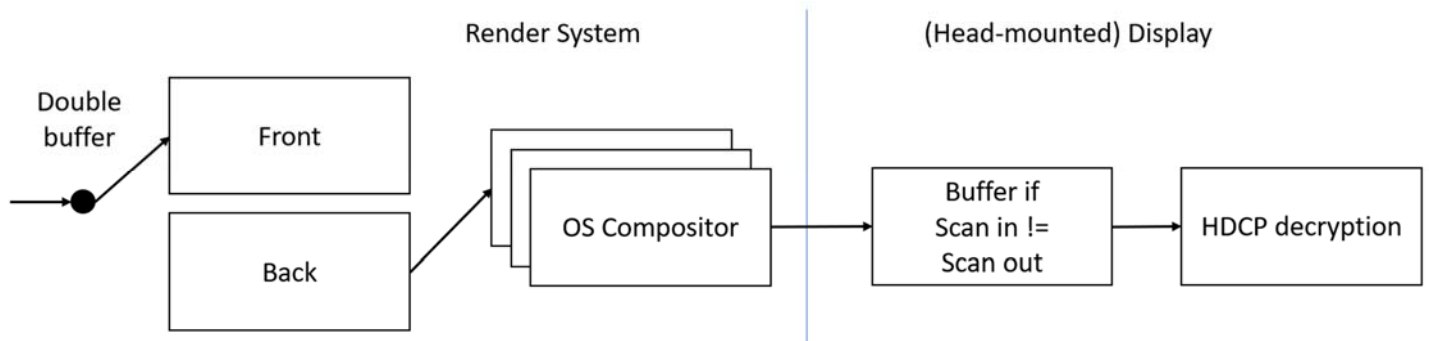
Display distortion correction: For head-mounted displays whose optics distort the display screen, a separate rendering pass is done onto a flat geometry that is warped to invert the optical distortion using the rectangular texture output from the previous stages. For flat-panel display, this pass is not needed. The delay is expected to take **~0-1ms**.

Synchronization Delays

If the different components in the system do not operate at exactly the same rate, or if their start times drift compared to each other (perhaps because of a variable duration on one or more stages), then the relative start times of each stage can change compared to the completion time of the previous stage. When this happens, the completion time for a previous stage can shift and happen just after the start time, causing a whole-frame delay between stages. When it shifts back, the latency is reduced; this shifting causes jitter in moving scenes, which is distracting.

Solution: To avoid this, all stages should be synchronized to run at the same rate and at relative phases that cause each stage to complete just before the following stage begins. This maximizes the time available to the previous stage while still meeting its deadline. This can require additional hardware timing signals with fixed delays to be sent to various parts of the system to cause them to properly align. It can also make use of feed-backwards information (time until scan-out from the GPU) within the rendering system along with busy-waiting on earlier stages.

Display



The diagram above shows the potential latency due to a multi-buffer pipeline. Each portion of the pipeline is present to optimize something other than latency, with the consequence of adding latency to the system. The OSVR RenderManagerⁱ removes all but one of the buffered-frame delays as described in VR Gemsⁱⁱ and as follows:

- Double buffering:** (Optimizes the length of time available to render a scene): To avoid scanning a partially rendered buffer to the display, and to maximize the time the application has available to render, two or more buffers are used in a round-robin fashion, with the application writing to one and the display scan-out hardware reading from the other. The application swaps buffers, blocking until the scan-out is completed. **Solution:** OSVR uses only a single render buffer, and it completes its final rendering pass in the time between the start of the display's vertical blanking interval and start of the next frame's scan out. This required vendor-specific calls that have since been moved into general OS interfaces.
- OS Compositor:** (Optimizes for animation, shadowing, and other effects): The window manager on the operating system provides composition functions to support various layering effects for the OS GUI. This requires one or more additional buffers to perform compositing in after all applications have displayed. **Solution:** OSVR uses *direct to display* mode rendering that removes the display from those used by the OS window manager and drives the display full screen directly. This required vendor-specific calls that have since been moved into general OS interfaces.
- HMD scan matching:** (Optimizes for flexibility in scan-out): Many head-mounted displays natively scan out from left to right and appear to the operating system as being much taller than they are wide. To enable applications to render at the orientation that the viewer will see, they offer to scan out top to bottom. To do this, they must buffer an additional frame. **Solution:** OSVR can be configured to run a display in its native orientation, and it performs the required rotations in the transformation matrix that it uses to render the final pass.
- HDCP (copyright) decryption:** (Optimizes for the protection of copyrighted content): To prevent people from capturing unencrypted data as it is sent to a display, the industry has implemented an encryption protocol for data sent to display devices. Although in theory this could be disabled for non-copyright content, some vendors' display drivers require it to be always enabled. This decryption requires buffering and adds latency, about a single frame time or **~16ms for 60fps display**. **Solution:** OSVR was unable to convince the GPU manufacturers to disable this feature or to allow the drivers to send to a non-HDCP display device. **The Army may be able to work with a vendor to disable this, bringing the latency down to 0ms.** OSVR's solution was to measure the end-to-end latency and to increase the prediction interval to include this additional latency. By using new head tracking data to adjust for rotations, this single-frame final rendering pass had small enough latency even with this additional frame.

Scan-out delay: On a progressive-scan display, the first line has almost no delay from render completion to display, but the last line in the image has a full frame time of delay. The average delay for a scan line is half of the frame time. For a **60fps display, this is ~8ms**.

Persistence delay: Each display pixel remains lit for some period of time. On some displays, this is very short. On others, this can be an entire frame time. The average presentation delay beyond initial display is half of this persistence time, which would be **0-8ms for a 60fps display rate** depending on the particular display's persistence.

Avoiding Sickness

It has long been known that latency is the major cause of misregistration in virtual reality (VR) and augmented reality (AR) displays. Rich Holloway's 1996 dissertation at UNC pointed out that for normal head motions in a nearby environment, every 1ms of total system latency added 1mm to offset error; this means that there is 1.6cm of error during just the 16ms scan out of a 60Hz display.ⁱⁱⁱ For a moving vehicle and further objects, this will be at least 10x worse. Ron Azuma's 1995 dissertation at UNC showed that by adding angular rotation rate detectors and accelerometers to an optical tracking system, the head pose at rendering time could be predicted out to about 80ms in the future^{iv} and Greg Welch's 1996 dissertation at UNC showed how to combine the inputs from multiple types of sensors to produce an optimal estimate of head pose and pose velocity.^v Marc Olano's 1995 I3D paper showed that this could be pushed to line-by-line adjustments of progressive-scan head-mounted displays.^{vi}

Rotational self-motion latency: These techniques and others can be used together to predict, over-sized render, and then fix up based on new tracker data to hide the effects of rotational self-motion latency. This is done in modern VR and AR toolkits. The OSVR RenderManagerⁱ system developed by ReliaSolve removes all but one of the buffered-frame delays as described in VR Gemsⁱⁱ (see Figure 1 on page 1) and it includes the "just in time pixels" approach not handled by other toolkits (Figure 2, below).

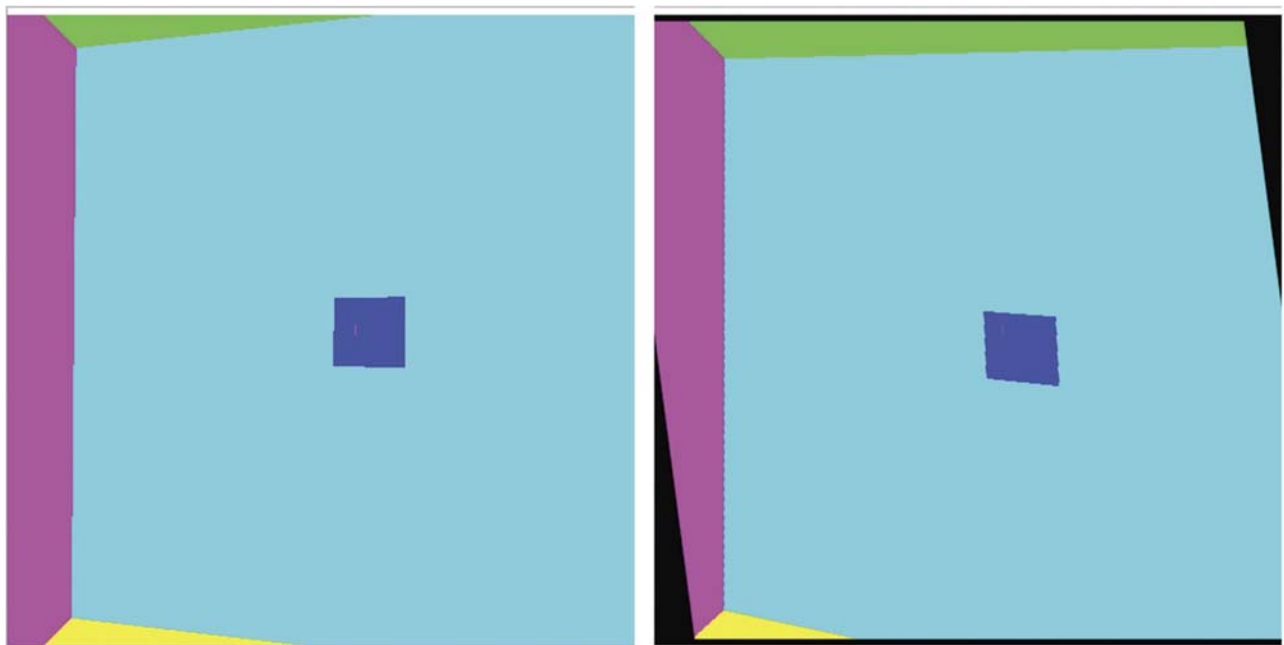


Figure 2: Just-in-time pixels: An image of a small dark blue cube within a colored cube room taken with the head held still shows the cube to be square. Right: A just-in-time rendering of this same scene with the head rapidly moving shows shearing in X and a scale reduction in Y to compensate for changes in location of the scan lines during scan-out. Note that the still image is deceiving because when you look at it on paper it's entirely "rendered" at the same time and your head isn't moving – when viewed from within the moving HMD, the scene appears normal; without just-in-time rendering, the cube appears to be stretched in Y and sheared in the opposite direction. The black gaps around the image can be avoided by rendering an oversized image before warping.

Translational self-motion latency: When the vehicle or user's head is moving, or when the user's display viewpoint is different from the camera's viewpoint, pixel motions in the scene depend on the distance to the object being viewed and not only on the angle of the user's viewpoint. Handling either of these cases requires an estimate of the depth of each pixel in the scene, which can be obtained by several approaches, including the following:

- As shown to the right, include two additional cameras (total of 23 vs. 21) whose centers of projection differ from the detail cameras and that together capture the entire field of view of the detail cameras. These can be offset in X and/or Y from the other cameras to avoid obstructed views. For each region of the detailed cameras, render multiple depths of the appropriate wide-field camera to find the best-aligned result.
- Include LIDAR sensors whose fields of view cover the range of the detail cameras, providing coarse depth estimates in all viewing directions.



Using these depth estimates, knowing the time at which they were taken and aircraft velocity, construct a surrounding low-resolution depth surface that has the high-resolution camera textures mapped onto it. Move this surface over time relative to the aircraft and then reproject it from the point of view of the pilot's display(s). This can also enable stereo display for the pilot, which drastically improves depth estimation when in close quarters with obstacles or targets.



Figure 3: Translational self-motion compensation: A helicopter flying towards an embankment causes a shift in pixel locations due to the angle changing to each point on the ground between the time the camera samples it and the display shows it. When the depth of each pixel is known, the difference in position between the helicopter at display time and sample time can be applied to the original pixel locations, shifting them to compensate for this motion.

“No swimming” When used together, these techniques counteract the swimming caused by system latencies. They make the environment surrounding the vehicle remain stable while the vehicle files and the pilot looks in different directions. Using over-sized rendering and repeating sensor measurements, ***self-motion latencies, including sensor integration and display persistence, can be entirely hidden.***

Potential Solutions

Consumer head-mounted display

Photon latency: **29-67ms (19-57 internal)**.^{*} Uncorrectable self-motion latency: **0ms**.

Directly driving a head-mounted display enables the correction of self-motion latency during the rendering phase using additional information (relative orientation tracking of the aircraft and head for rotational self-motion; relative position tracking of the aircraft and the means to estimate pixel depth from the camera for translational self-motion). The buffering and processing required to display to this device and to perform the correction increases the photon latency but enables the self-motion latency to be corrected.



Pros:

- Minimum corrected self-motion latency. With low-noise sensors, can be completely removed.
- Wide field of view, providing situational awareness and aiding in navigation and search.

Cons:

- Correction of translational self-motion latency requires depth estimation, which may require additional hardware. (This is not needed when hovering or moving slowly.)
- Varjo displays have 3 additional frames of latency (at 120 Hz) in their OpenXR driver.

Elbit Xsight lab kit

Photon latency: **43-65ms (33-55 internal)**.[†] Uncorrectable self-motion latency: **0ms**.

This is a complete system that includes scan-in and rendering. This device sends a stream of requests for camera orientations relative to the aircraft and receives a stream of video frames at **50fps**. The internal interfaces within the device require buffering (1/2 of a 50Hz frame) plus an intermittent additional frame due to synchronization delays between the 60Hz input and 50Hz output. This adds 10-30ms to the total system latency beyond those described above. However, by gen-locking the HMD to the graphics card and running the cameras at 50Hz we should be able to remove the intermittent additional frame, reducing the latency to 10ms.



The image requests include request time and by using their interface that embeds timing information into the displayed frames, we were able to **correct all self-motion latency**.

Pros:

- Flight-certified solution that is already in the field for other applications.

Cons:

- Narrower field of view than other HMDs.
- Higher photon latency than other solutions when using demo unit. **Note:** This will be improved when using the production system, which runs at 60-120 Hz rather than 50Hz.

^{*} Photon: 10ms integration, 8ms buffering, 1ms transmission, 0-5ms depth, 1-5ms composition, 1-5ms rendering, 0-1ms undistortion, 0-16ms HDCP, 8ms scan-out, 0-8ms persistence.

[†] Photon: 10ms integration, 10ms buffering, 1ms transmission, 0-5ms depth, 1-5ms composition, 1-5ms rendering, 0-1ms undistortion, 10ms buffering, 10ms scan-out, 0-8ms persistence.

Display per camera

Photon latency: **10-18ms (0.01 internal)**, Uncorrectable self-motion latency: **10-18ms**.

Directly driving a display from each camera could produce sub-millisecond latency from camera scan-out to display, assuming lenses without distortion. By selecting displays whose scan-out matches the scan-out direction of the cameras, this presents each line out on the display as soon as it leaves the sensor.

Pros:

- Minimum possible photon latency, display at sensor scan-out time + <1ms.
- Maximum available field of view at full resolution.
- Very simple electronics with no software for see-through.
 - Simple electronics and software to combine heads-up instrument data.

Cons:

- Very intrusive hardware.
- Very strong dissonance between eye-convergence/stereo perceived depth and actual scene depth.
 - This will **require covering one eye**, producing a monocular view of the scene.
- Displays may be too close to focus on, requiring contact lenses or glasses to move virtual images further away.
- Cannot remove self-motion latency caused by sensor integration and display persistence.
- Cannot shift center of projection from the nose of the aircraft to pilot's point of view.



ⁱ <https://github.com/ReliaSolve/OSVR-RenderManager>

ⁱⁱ <https://www.reliasolve.com/index.php/vr-tricks/vr-concepts-illustrated-using-osvr/>

ⁱⁱⁱ <https://techreports.cs.unc.edu/papers/95-016.pdf>

^{iv} <https://techreports.cs.unc.edu/papers/95-007.pdf>

^v <https://techreports.cs.unc.edu/papers/96-051.pdf>

^{vi} Marc Olano, Jon Cohen, Gary Bishop, "Combatting Rendering Latency," Proceedings of the 1995 Symposium on Interactive 3D Graphics (I3D '95), Monterey, California. pp. 19-24. 1995.